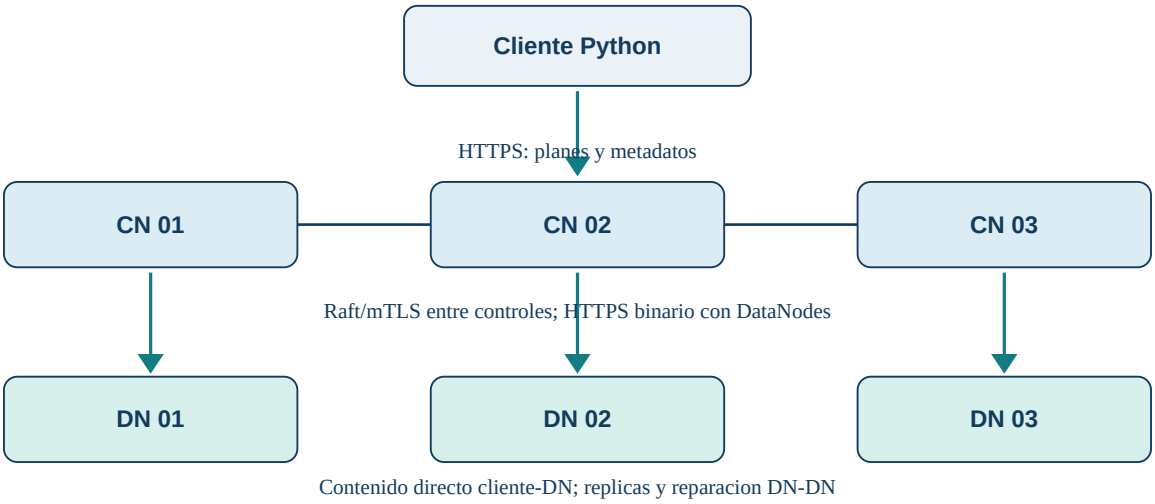
DFSha

Sistema de archivos distribuido

Alta disponibilidad, rendimiento y seguridad



Informe técnico final - Proyecto 1

Andrés Vélez Rendón
Jose Luis Restrepo

Universidad EAFIT
ST0263 / SI3007 - Semestre 2026-2
24 de octubre de 2026

1. Introducción

DFSha almacena archivos binarios mediante bloques inmutables repartidos y replicados entre servidores. El cliente presenta un árbol de directorios y obtiene del servicio las ubicaciones actuales de los bloques. El plano de control administra metadatos; el plano de datos transporta y conserva el contenido. Esta separación permite transferir bloques de manera concurrente y evita que los archivos completos atraviesen el coordinador.

La implementación continúa el proyecto original del equipo: servidores en Go, cliente de línea de comandos en Python, APIs REST y despliegue mediante Docker. El código de GitHub contenía una evolución del Hito 2 posterior al ZIP suministrado; se tomó esa evolución como base para preservar sus módulos, pruebas, algoritmos y procesamiento acotado por bloques. La revisión incluyó el enunciado de siete páginas, la especificación original del Hito 1 de dieciséis páginas y la documentación y fuentes del Hito 2.

2. Problema y alcance

Un único servidor de archivos concentra capacidad, tráfico y riesgo de fallo. Replicar un archivo completo en cada máquina reduce la capacidad útil y desaprovecha el paralelismo dentro de un archivo. DFSha divide el contenido en bloques, mantiene varias copias de cada bloque y conserva un mapa persistente de su ubicación.

La opción asignada es **Cliente/Servidor con un servicio compuesto por servidores dentro de una red privada**, patrón Maestro-Trabajador. El cliente es externo al servicio. Tres ControlNodes coordinan metadatos mediante consenso; al menos tres DataNodes constituyen la configuración de referencia. Un cuarto DataNode permite demostrar crecimiento sin cambiar las direcciones de archivos configuradas en el cliente.

El servicio ofrece archivos y directorios privados por usuario, transferencia completa y primitivas de acceso. No monta una unidad del sistema operativo, no proporciona semántica POSIX completa y no incluye ACL para compartir directorios entre usuarios. La sustitución de una versión conserva bloques inmutables y publica de una sola vez los nuevos metadatos. La pérdida de todas las copias válidas de un bloque impide recuperar ese archivo.

3. Objetivos

3.1 General

Implementar y documentar un sistema de archivos distribuido por bloques que combine almacenamiento y acceso concurrentes, persistencia, replicación, control de acceso y recuperación ante los fallos declarados.

3.2 Específicos

1. Mantener una jerarquía privada por usuario y una CLI para transferir y consultar archivos.

2. Calcular dinámicamente el tamaño de bloque y distribuir réplicas en nodos diferentes considerando su ocupación y capacidad.

3. Conservar metadatos confirmados ante el fallo de un ControlNode mediante un registro persistente replicado por mayoría.

4. Detectar fallos de DataNodes, aprovechar réplicas disponibles y reconstruir redundancia entre servidores.

5. Proteger credenciales, autorizaciones, comunicaciones y contenido almacenado mediante mecanismos criptográficos estándar.

6. Proporcionar pruebas automatizadas, procedimientos de despliegue y evidencias reproducibles, distinguiendo implementación de validación efectivamente ejecutada.

4. Marco teórico

Un DFS separa el nombre lógico de un archivo de su ubicación física. En un diseño por bloques, un archivo es una secuencia ordenada de identificadores y tamaños. Las réplicas permiten recuperar un bloque desde más de un servidor; SHA-256 identifica su contenido y permite detectar cambios durante almacenamiento o transferencia.

El consenso resuelve el orden de las decisiones del plano de control. Raft mantiene un registro replicado, elige un líder por términos y confirma entradas mediante mayoría. Estas reglas ofrecen una base para conservar decisiones confirmadas a pesar de fallos de parada y particiones; la disponibilidad requiere una mayoría conectada. Se adopta la implementación Go de HashiCorp. Referencias: artículo de Ongaro y Ousterhout, implementación HashiCorp Raft.

La confidencialidad del contenido en disco se implementa mediante AES-GCM, un modo de cifrado autenticado disponible en la biblioteca estándar de Go. Las contraseñas se comprueban contra hashes bcrypt con sal. Los tokens y las autorizaciones temporales de bloques utilizan HMAC; no sustituyen TLS, porque quien posea una autorización vigente puede usarla dentro de su alcance. Referencias: Go crypto/cipher, Go bcrypt.

5. Requisitos y decisiones preservadas

Grupo del enunciado	Solución adoptada
RF1: gestión del sistema de archivos	`ls`, `cd`, `pwd`, `mkdir`, `rmdir`, `rm`, `mv` y `stat`.
RF2: transferencia	`put`/`send` y `get`/`receive`/`recv`.
RF3: acceso	`open`, `close`, `read`, `write` y `lock`, con handles y reserva de escritor.
RNF1: escalabilidad	Alta dinámica de DataNodes, bloques y transferencias concurrentes; limitaciones de metadatos declaradas.
RNF2: disponibilidad	Tres CN con mayoría y varias réplicas de bloques.
RNF3: consistencia	Registro replicado, bloques inmutables, reserva y publicación atómica.
RNF4: particionamiento	Archivos multibloque repartidos y réplicas sin duplicar DataNode.
RNF5: rendimiento	Cuatro bloques simultáneos por defecto y atención HTTP concurrente.
RNF6: seguridad	Autenticación, árbol privado, TLS, autorización de bloques y AES-GCM.
RNF7: transparencia	Rutas lógicas y planes dinámicos, sin direcciones de bloques fijadas en el cliente.
RNF8: ajustes	Desviaciones respecto del Hito 1 justificadas en este informe.

Se conservan el rango de bloques de 8-64 MiB, `k=4`, RF de referencia 2, heartbeat de DataNodes cada 3 s, caída tras tres intervalos perdidos, reporte cada 30 s, leases de escritura de 60 s, umbral de ocupación del 90 % y rebalanceo por diferencia superior al 10 %. Son parámetros configurables; las variables concretas y sus unidades se documentan en el README.

6. Arquitectura e implementación

6.1 Cliente

La CLI Python utiliza bibliotecas estándar para HTTPS, concurrencia, archivos y hashes. Conoce una lista de ControlNodes para localizar al líder. Obtiene las direcciones de DataNodes en cada plan y trabaja directamente contra ellos. Guarda sesión, directorio actual y handles en un archivo local configurable mediante `DFSHA_SESSION`; el token no se imprime como parte de la salida de autenticación.

Cada worker trabaja con un bloque y la descarga se escribe por desplazamiento sobre un archivo `.part`. El nombre final se publica localmente después de verificar todo el contenido. La memoria depende del número de workers y del tamaño de bloque, además de los metadatos del plan; no del tamaño completo del archivo.

6.2 ControlNodes

Los módulos existentes de configuración, namespace, planificación, subida, descarga y respuestas HTTP se extienden con persistencia y consenso. Un estado lógico incluye directorios, archivos, reservas, handles y ubicaciones. Cada modificación se aplica mediante Raft y requiere mayoría. Los registros se conservan con BoltDB y se generan snapshots para recuperar el estado.

Para simplificar la implementación educativa, cada comando replica una representación completa del estado lógico. Una revisión CAS se valida dentro de la máquina de estados: un líder suspendido que recupera el liderazgo no puede reemplazar decisiones confirmadas con una copia antigua. Esto facilita mantener una transición atómica y comprender su recuperación, pero consume tráfico y CPU proporcionales al volumen de metadatos. Se declara como límite frente a comandos incrementales y namespace particionado.

Las lecturas autoritativas se sirven desde el líder después de verificar su liderazgo y aplicar las entradas pendientes. Un seguidor responde con indisponibilidad temporal e información del líder cuando lo conoce. El cliente prueba otros controles; no interpreta un seguidor atrasado como fuente equivalente de una lectura vigente.

6.3 DataNodes

Cada DataNode expone su propia API, guarda bloques cifrados, verifica contenido y atiende transferencias simultáneas. Las operaciones internas requieren autenticación del clúster. Un cliente no obtiene permiso para leer cualquier bloque por tener una sesión válida: necesita la autorización temporal emitida para el recurso y operación correspondientes.

Registro, heartbeat e inventario permiten al líder actualizar topología y copias. Los inventarios permiten detectar archivos ausentes o dañados y conciliar ubicaciones. Los bloques temporales incompletos no se publican como bloques válidos.

7. Particionamiento, distribución y replicación

7.1 Tamaño y orden

Para un archivo de tamaño `S` y `N` DataNodes vivos, el tamaño se calcula como:

```
B_propuesto = S / (k × N)
B = clamp(8 MiB, potencia_de_dos_inferior(B_propuesto), 64 MiB)
n_bloques = ceil(S / B)
```

Se trata el archivo vacío explícitamente. El último bloque puede ser menor que `B`. El cliente calcula SHA-256 sobre el contenido en claro; el identificador no depende del nonce o del cifrado físico de cada réplica. Los metadatos conservan índice, tamaño, hash y ubicaciones para reconstruir el orden exacto.

7.2 Colocación y capacidad

El líder escoge los nodos elegibles de menor ocupación relativa. La carga virtual incorpora las asignaciones del propio plan para evitar que todos los bloques se dirijan a los mismos destinos. No se asignan dos copias del mismo bloque al mismo DataNode. La admisión debe considerar las reservas activas y la capacidad necesaria por destino; un plan sin espacio devuelve un error de almacenamiento insuficiente.

Se conserva la heurística del Hito 1, inspirada en asignación voraz. La cota clásica de Graham no se utiliza como garantía de este sistema: medir porcentajes sobre capacidades distintas y exigir ubicaciones diferentes para las réplicas cambia las hipótesis del problema clásico.

7.3 Factor de replicación

La referencia son dos copias por bloque con tres DataNodes. Se conserva la restricción operativa del Hito 1 de reservar un nodo como posible destino de reparación, limitando el objetivo según la topología. Si disminuye el número de nodos vivos, la especificación permite guardar menos copias y completar la redundancia después.

Una escritura degradada con una copia sigue siendo recuperable mientras esa copia exista, pero **no tolera otra pérdida de ese bloque** antes de repararse. El estado degradado se hace visible en metadatos y registros; no se presenta como RF 2 satisfecho.

El 99,5 % del Hito 1 es un objetivo de disponibilidad. Su cálculo ilustrativo presupone disponibilidad independiente de nodos y, al multiplicar disponibilidad por bloque, independencia adicional entre bloques. Los bloques que comparten máquinas o dominios de fallo están correlacionados; por ello no se utiliza aquella fórmula como certificación de disponibilidad ni se afirma un porcentaje observado.

7.4 Reparación y balanceo

Una pérdida de heartbeat excluye al nodo de nuevos planes. Si un bloque conserva una copia válida, el líder ordena copiarla desde ese DataNode hacia otro elegible. El cliente no tiene que volver a cargar el archivo.

El rebalanceo se activa cuando la diferencia de ocupación supera el umbral configurado. La secuencia segura es copiar, verificar el nuevo destino, confirmar sus metadatos en mayoría y retirar la ubicación sobrante. La eliminación física espera el margen de autorizaciones vigentes. La reparación y el rebalanceo tienen trabajo limitado por ciclo para controlar el tráfico; no se presentan como un optimizador global de colocación.

8. Consistencia y acceso

8.1 Subida y publicación

Un plan reserva una ruta para un escritor durante 60 segundos por defecto. El archivo nuevo permanece invisible mientras los bloques se transfieren. Antes del commit, el control consulta a los destinos mediante HEAD autenticado para comprobar tamaño e integridad de los bloques del plan. El DataNode descifra y verifica localmente; los bytes del archivo no atraviesan el ControlNode.

El commit publica metadatos completos y libera la reserva en una transición replicada. Un hash de archivo completo informado por el cliente permite comprobar la reconstrucción final, pero el control no recalcula dicho hash recibiendo todo el archivo; valida bloques y tamaños en sus servidores. Las operaciones que no llegan a commit no se convierten en archivos publicados.

8.2 Primitivas de acceso

Primitiva	Semántica de esta entrega
`open PATH --mode r`	Obtiene un handle temporal para la versión consultada; no descarga el archivo completo.
`open PATH --mode w / lock PATH`	Reserva un único escritor y devuelve un handle temporal.
`read PATH OFFSET LENGTH`	Pide solo los bloques intersectados y emplea `Range` para el intervalo útil. Puede escribir binario en stdout o en un archivo.
`write HANDLE LOCAL --offset N`	Construye localmente una versión nueva, inserta los bytes indicados y publica por copia sobre escritura. No modifica bloques existentes.
`close HANDLE`	Cierra y libera la sesión. La publicación requiere previamente completar la subida y su commit.

La escritura con offset es una comodidad de la CLI: puede descargar y volver a subir el archivo entero para crear la versión nueva. No equivale a modificación parcial en DataNodes. El archivo visible cambia atómicamente y las copias antiguas permanecen inmutables hasta su recolección segura. `put` y `get` también pueden componerse con handles y sesiones explícitas.

`cd` es estado del cliente: valida el directorio contra el servicio y conserva la ruta actual. Los permisos se comprueban en el servidor; cambiar el archivo local de sesión no permite acceder al árbol de otro usuario.

8.3 Interrupciones, reintentos y limpieza

Los identificadores de bloque basados en contenido permiten reintentar un PUT sin duplicar el mismo contenido. Las reservas y handles tienen vencimiento; una operación vencida debe obtener una autorización nueva. Un error de red posterior a enviar un commit puede dejar un resultado ambiguo para el cliente, por lo que debe consultar el estado antes de iniciar otra escritura. Los contratos concretos de reintento están en la documentación de API.

La limpieza compara referencias de archivos publicados, reservas y sesiones vigentes, espera un periodo de seguridad y después ordena eliminar huérfanos. Borrar metadatos y borrar inmediatamente cualquier hash sería incorrecto: un mismo bloque puede estar referenciado por varios archivos o por una lectura aún autorizada.

9. Alta disponibilidad y desviación del Hito 1

El Hito 1 propuso elección del matón modificada por actualidad de metadatos y mayoría. Esa propuesta selecciona un coordinador, pero no especifica todas las reglas para votos, términos, registros divergentes y confirmaciones bajo partición. La mayoría por sí sola no demuestra que un líder futuro conserve cada operación confirmada.

Se reemplaza ese mecanismo por una biblioteca de consenso Raft, manteniendo los tres ControlNodes y el diseño del servicio. La aplicación sigue siendo propia: namespace, planes, validaciones, API y gestión de bloques; la biblioteca resuelve el registro y la elección. El equipo debe sustentar qué aporta la biblioteca y qué decisiones pertenecen a DFSha.

Aspecto	Hito 1	Entrega final y razón
Elección	Matón modificado	Raft con términos, votos y registro persistente para evitar inventar consenso.

Aspecto	Hito 1	Entrega final y razón
Transporte CN↔CN	REST/JSON	Transporte de la biblioteca Raft con TLS mutuo, puerto 9000. Las APIs de cliente y datos conservan REST/HTTPS.
Lectura de seguidores	Permitida	Se exige líder y comprobación de mayoría para evitar lecturas atrasadas presentadas como actuales.
Autorización de bloques	Token de sesión global	Capabilities de vida corta con alcance de recurso y operación; disminuyen el acceso directo indiscriminado.
Permisos compartidos	Posibilidad de conceder permisos a directorios	Árbol privado por propietario. ACL de compartición no implementada.
Parámetros	Archivo de configuración	Variables y archivos externos de secretos; se cambian sin recompilar.

Tres controles permiten continuar con dos conectados. Sin mayoría se rechazan operaciones autoritativas y nuevas publicaciones. El reinicio recupera registros y snapshots persistentes; no se borra el directorio de metadatos para «resolver» un fallo. Se presupone almacenamiento que respeta sincronización a disco y fallos de parada, no nodos maliciosos ni pérdida simultánea de una mayoría de discos.

10. Seguridad

- Las contraseñas no se almacenan en claro: cada ControlNode lee la misma configuración de usuarios con hashes bcrypt. La incorporación de un usuario requiere actualizar esa configuración de forma coherente y reiniciar según la guía; los usuarios no se administran mediante una API pública improvisada.

- Las sesiones están firmadas y expiran; el valor de referencia es una hora. La sesión identifica al propietario del árbol lógico. No hay contraseña ni token universal de desarrollo como mecanismo de acceso final.

- Los DataNodes reciben autorizaciones temporales firmadas con alcance de nodo, recurso y método. Leer o escribir un hash conocido requiere autorización; borrar y reparar son operaciones internas.

- Antes del commit, cada DataNode debe acreditar que esa cuenta presentó los bytes del bloque mediante un PUT verificado. La autoría se conserva por bloque/cuenta; conocer un hash ajeno o que exista físicamente no concede propiedad. Las reparaciones internas no conceden permisos a otras cuentas.

- Las comunicaciones REST usan HTTPS y validación de CA. El consenso CN↔CN exige certificados de cliente mediante TLS mutuo. La confianza del clúster se complementa con secretos compartidos para sus APIs internas.

- Cada bloque se conserva con AES-GCM, nonce aleatorio e integridad autenticada. Una modificación del ciphertext se detecta al descifrar y el hash del contenido vuelve a comprobarse.

- Las claves y certificados se generan fuera de las fuentes y se excluyen de Git. Copiar la configuración privada entre nodos forma parte del aprovisionamiento. No se incluye gestión con KMS, rotación en línea ni cifrado de metadatos a nivel aplicación; se recomienda volumen cifrado para estos últimos.

El modelo confía en los administradores y nodos autenticados del clúster. Una clave compartida comprometida amplía el alcance del atacante; no se afirma seguridad frente a servidores internos maliciosos. El modo que desactiva verificación TLS sirve únicamente para pruebas explícitas y no demuestra el RNF6.

11. Despliegue

El arranque reproducible local se realiza mediante la guía raíz, Dockerfiles, Compose, persistencia y secretos generados. Hay tres procesos de control, tres procesos de datos y cliente; sus peticiones cruzan sockets de red. Esto demuestra comunicación entre componentes, pero no prueba separación de fallos físicos cuando todos viven en una computadora.

Para AWS Academy se conserva la distribución de una pareja CN/DN en cada una de tres EC2. Los volúmenes de metadatos y bloques deben persistir al recrear contenedores. El puerto 9000 se limita a los controles; 8000 y 8001 se habilitan solo para orígenes autorizados dentro de red privada o VPN. El cliente necesita alcanzar tanto el ControlNode como los DataNodes anunciados. Exponer solo el control sin ruta a los DataNodes no basta.

Los certificados deben incluir los nombres o direcciones efectivamente anunciados. Las direcciones privadas de cada VM se configuran de forma coherente y se conserva el mismo material de confianza. Un cuarto DataNode se registra dinámicamente y se añade sin modificar rutas de archivos del cliente. Los comandos de instalación, variables, grupos de seguridad, limpieza y recuperación se encuentran en deploy/aws/README.md.

12. Pruebas y resultados

La validación debe registrar pruebas ejecutadas sobre esta entrega, con comandos, entorno y resultados observados. La existencia de un test o una configuración no acredita su ejecución. La matriz TRAZABILIDAD.md enlaza las evidencias finales; los resultados automatizados generados por los scripts constituyen el detalle reproducible.

Batería	Qué debe demostrar	Estado de evidencia de este informe
Unitaria Go/Python	Planes, reservas, validaciones, almacenamiento, transporte y cliente.	35 pruebas Go y 31 Python aprobadas; go vet aprobado. Go probado en Windows y Linux (contenedor Go 1.26).
Funcional integrada	Usuarios, namespace, archivo vacío y multibloque, hashes, rangos, handles y COW.	Integración real aprobada para usuarios, archivo vacío, multibloque, rangos, handles, COW y publicación atómica.
Distribución	Bloques y réplicas en DN distintos; capacidad y ocupación.	24 MiB: 3 bloques, RF 2, 2 bloques por cada uno de 3 DN; ningún DN conserva el archivo completo en ese ensayo.
Fallos	Caída de DN, otra réplica, corrupción, líder caído, falta de mayoría, reinicio e interrupción.	Aprobadas caída DN, corrupción y reparación, cambio de líder, reinicio completo y rechazo sin mayoría. Evidencias en JSON.
Escalado y rendimiento	Alta de DN, reparación/balance, tamaños y workers 1/4.	Cuarto DN incorporado y rebalanceado. Transferencias de 1/24 MiB, workers 1/4, medidas y verificadas con SHA-256.
Docker	Construcción de imágenes, healthchecks y ejecución desde cero.	Imágenes construidas, 6 servicios saludables y demostración real aprobada con Python 3.13/TLS estricto; 9 checks.
AWS e Internet	Tres VMs, dominios de fallo, red y certificados reales.	Requiere ejecución en cuenta académica; no se infiere de la prueba local.

No se publican porcentajes de disponibilidad, mejoras de rendimiento ni tiempos inventados. Las duraciones de una prueba local describen únicamente ese equipo, tamaños y configuración; no se extrapolan a EC2 sin medir. La grabación del video es un entregable distinto del guion.

Mediciones registradas

Las siguientes mediciones proceden de `docs/resultados/integration.json`, en procesos locales Windows con TLS. Se midieron mientras también había servicios Docker y otra batería activa; describen ese entorno y no son una comparación controlada de EC2. Cada descarga verificó SHA-256.

Tamaño MiB	Workers	Subida s	Descarga s
1	1	0.2101	0.0570
1	4	0.2991	0.0703
24	1	1.0091	0.3169
24	4	0.3303	0.4736

La batería final aprobó 15 escenarios en 41.42 s. La extendida aprobó 16 escenarios en 265.49 s, incluidos renovación automática, reserva vencida, GC físico y retirada segura hasta RF 2. Los resultados están separados porque la ejecución extendida precedió al refuerzo final de autoría de bloques; ese refuerzo está cubierto por la batería final y las unitarias. Todos sus procesos temporales quedaron cerrados.

Docker aprobó 9 comprobaciones con tres CN y tres DN saludables, Python 3.13 y validación TLS estricta. Su archivo de 24 MiB tardó 0.4348 s en subir y 0.2206 s en descargar. El informe conserva estas medidas sin atribuirles disponibilidad anual.

13. Limitaciones y trabajo futuro

1. El consenso exige mayoría de los tres controles; dos controles caídos o aislados bloquean operaciones autoritativas.

2. RF 2 tolera la pérdida de una copia de cada bloque mientras la otra conserve contenido válido. Una escritura degradada puede reducir esa tolerancia a cero.

3. Replicar el estado completo por modificación facilita la implementación, pero limita el número de archivos y operaciones de metadatos por segundo.

4. Las unidades de concurrencia y cifrado son bloques. La memoria se acota por tamaño de bloque y peticiones concurrentes; no se afirma memoria constante respecto de concurrencia ilimitada.

5. La CLI con `write --offset` puede transferir una versión completa para modificar unos bytes. No hay escritura remota de bloques mutable ni montaje POSIX.

6. No hay compartición por ACL, grupos, 2FA, KMS, rotación en línea, interfaz gráfica ni tolerancia bizantina.

7. Las credenciales de usuario son configuración replicada administrativamente entre controles, no una base dinámica de usuarios gestionada por Raft.

8. Los reportes, heartbeats y barridos introducen intervalos de detección y convergencia. El rebalanceo es gradual.

9. La disponibilidad anual requiere monitorización y medición prolongadas. El 99,5 % se conserva como objetivo, no como resultado demostrado.

10. El despliegue público y la grabación deben realizarse y anexarse cuando exista acceso a la infraestructura correspondiente.

14. Conclusiones

La separación entre metadatos y contenido permite conservar una arquitectura distribuida explícita: el cliente obtiene decisiones del líder y transfiere bloques directamente con DataNodes. Los identificadores por contenido, la publicación atómica y los controles de réplica reducen la superficie de inconsistencia.

La alta disponibilidad de metadatos exige ordenar y persistir decisiones, además de elegir un servidor. La sustitución justificada del matón por Raft y el rechazo de operaciones sin mayoría hacen explícito el límite de tolerancia. La redundancia de bloques complementa esa garantía, pero su efectividad depende de conservar copias válidas en dominios de fallo adecuados.

La entrega se evalúa con evidencia reproducible y límites declarados. El código, la documentación y los procedimientos permiten revisar cada requisito y repetir las pruebas; un despliegue en VMs y el video deben aportar su propia evidencia de ejecución.

15. Referencias

1. Universidad EAFIT. *ST0263/SI3007, Proyecto 1: DFSha*, enunciado 2026-2. Documento entregado en el repositorio.

2. Vélez Rendón, A.; Restrepo, J. L. *DFSha: Especificación del proyecto, Hito 1*. 13 de septiembre de 2026. Documento original.

3. Vélez Rendón, A.; Restrepo, J. L. *Hito 2: Arquitectura distribuida y comunicaciones*. Informe conservado.

4. Ongaro, D.; Ousterhout, J. *In Search of an Understandable Consensus Algorithm, Extended Version*. Artículo.

5. HashiCorp. Implementación Raft para Go.

6. Go. Documentación de `crypto/cipher` y bcrypt.

Las referencias web anteriores son fuentes técnicas; las decisiones, límites y resultados del proyecto se verifican contra el código y sus reportes de pruebas.